

Design and Software Implementation of a Stellarium-Integrated Automated Telescope Control System

Author(s): Ghaith Zaghlouleh

Department of Physics, Faculty of Science, Damascus University, Syria

Department of Communication Technology, Syrian Virtual University, Syria

Email: ghaith.zaghlouleh@gmail.com

(ORCID: 0009-0004-1631-479X)

ARTICLE INFO:

Keywords:

Telescope
Automation,
Stellarium API,
Celestial Tracking,
Alt-Azimuth Control,
Python, Software
Design, Ground-
Based Instruments,
Amateur Astronomy,
System Architecture.

ABSTRACT:

Accurate celestial tracking is fundamental for astronomical observation and data acquisition. Manual telescope operation, however, presents limitations in efficiency and precision, particularly for extended observations or faint objects. This paper details the design and software implementation of a novel automated telescope control system that leverages the Stellarium planetarium software's API. The proposed system integrates real-time astronomical data from Stellarium with a flexible Python-based control application, designed to command a conceptual automated Alt-Azimuth telescope mount. The system architecture encompasses modules for Stellarium API communication, real-time coordinate acquisition (RA/Dec, Alt/Az), and a robust hardware interface manager (BoardManager) employing a defined serial command protocol for motor control. Control logic for converting target celestial coordinates into mount-specific pointing commands is presented. Simulation results, using Stellarium's real-time data output, demonstrate the software's capability to generate accurate pointing commands. It provides a practical and cost-effective alternative to existing astronomical control systems, enabling real-time Alt-Azimuth telescope tracking with modular hardware integration, suitable for both professional observatories and independent astronomers in resource-limited environments. While the physical hardware implementation is conceptual at this stage, this work highlights a modular and accessible software framework with potential applications in educational astronomy, amateur observatories, and as a foundation for developing low-cost, precise ground-based tracking instruments.

1. Introduction

The real-time tracking of celestial objects is paramount for effective astronomical observation and astrophotography, enabling detailed study of planets, stars, nebulae, and galaxies [1]. **Beyond fundamental astrophysics, precise ground-based optical observations are crucial for various astronomical applications, including the tracking of satellites and space debris, validation of orbital elements for spacecraft, characterization of near-Earth objects (NEOs) that may pose a hazard, and providing contextual astronomical data for space missions.** Manual telescope pointing, while foundational, can be time-consuming and challenging, especially for locating faint objects, tracking fast-moving near-Earth targets, or maintaining an object within the field of view for extended periods or imaging. Automated "GoTo" systems significantly mitigate these limitations by providing computer-controlled slewing and tracking capabilities [2].

Several software solutions and platforms facilitate telescope control. Planetarium programs like Stellarium [3] and SkySafari offer rich databases and sky visualization. Standardization efforts like ASCOM (Astronomy Common Object Model) [4] provide a framework for interoperability between software and hardware. However, developing custom, adaptable, and cost-effective automated telescope systems, particularly for upgrading existing manual instruments, for educational purposes, **or for establishing distributed networks of small aperture tracking instruments for space situational awareness (SSA) or transient event follow-up, remains an area of active interest.**

This paper is motivated by the need for an open, Python-based platform that tightly integrates with the readily available and powerful Stellarium software to control custom-designed telescope hardware. The proposed "InterstellarTracker" system (name used for the software component) focuses on the software architecture and control logic required to translate Stellarium's celestial data into actionable commands for a conceptual automated telescope mount. **Such a system can serve as an accessible testbed for developing algorithms and techniques applicable to more sophisticated ground-based optical tracking systems relevant to astronautics.** The objectives of this paper are to:

1. Describe the system architecture for a Stellarium-integrated telescope control system.
2. Detail the software implementation, focusing on API communication, control algorithms, and the hardware command protocol.
3. Present simulation results demonstrating the software's command generation capabilities.
4. Discuss the potential of this framework for future development of physical automated telescope mounts **with applications in astronautical support.**

The scope of this paper primarily covers the software design and simulated testing, with the physical hardware mount being a conceptual element for which the software is designed.

2. System Architecture

The proposed automated telescope control system is designed around three primary interacting components: the Stellarium planetarium software, the InterstellarTracker control software, and a conceptual microcontroller-driven telescope mount. A block diagram of the overall system is shown in **Figure 1**.

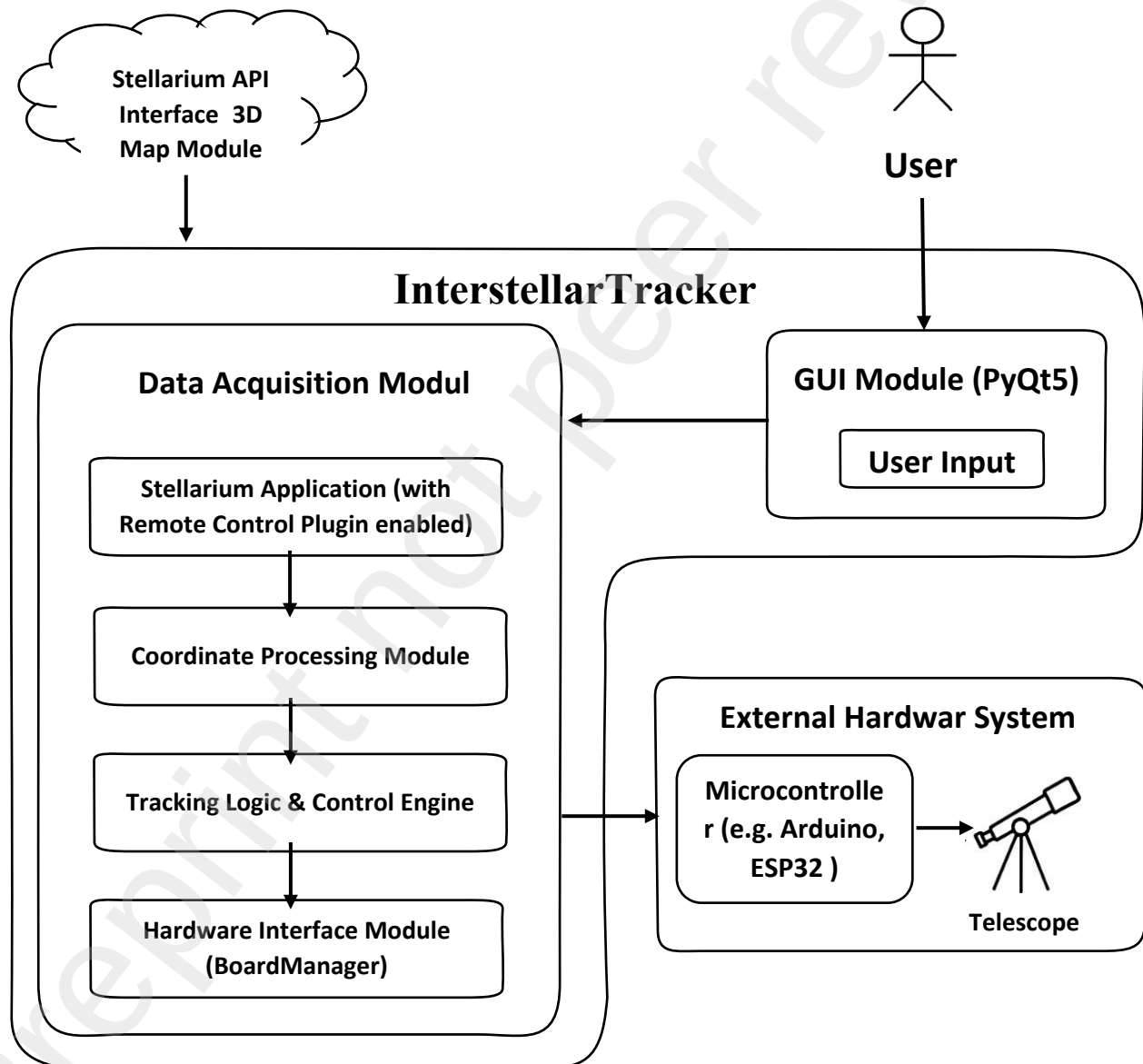


FIGURE 1: Conceptual block diagram of InterstellarTracker software modules and its interaction with Stellarium and external hardware.

2.1. Stellarium Planetarium Software

Stellarium serves as the primary source for celestial object data and as the user's interface for selecting targets. Its Remote Control plugin exposes an HTTP-based API, allowing external applications to query object positions (Right Ascension/Declination (RA/Dec), Altitude/Azimuth (Alt/Az)), observer location, time, and to control Stellarium's view [3].

2.2. InterstellarTracker Control Software

This Python-based application forms the core logic connecting Stellarium to the telescope hardware. Its modules include:

- **Stellarium API Integration Module:** Manages asynchronous communication with the Stellarium API to fetch real-time data for the currently selected object in Stellarium.
- **Real-time Position Data Acquisition:** Retrieves topocentric Alt/Az coordinates directly from Stellarium. If only RA/Dec are available for a specific query, this module (or the MCU firmware in a more advanced design) would be responsible for the transformation to Alt/Az based on observer location and time, although Stellarium typically provides both.
- **Coordinate Transformation Handling:** For an Alt-Azimuth mount, the topocentric Alt/Az coordinates provided by Stellarium can be used directly as pointing targets. This module ensures these values are correctly formatted. For future equatorial mount support, more complex transformations would be handled here or in the mount controller.
- **Motor Control Protocol Concept (BoardManager):** This software module, detailed in Section 3, defines and implements the communication protocol (primarily UART serial) for sending pointing commands to the microcontroller. It is designed for flexibility to support different microcontrollers and basic communication interfaces (UART, and foundational I2C/SPI).
- **User Interface (GUI/Telemetry Console):** A PyQt5-based GUI allows the user to configure hardware connection parameters (port, baud rate, board type), initiate/terminate the connection to the telescope hardware, start/stop tracking of the object selected in Stellarium, and view system logs and status. On Windows, it can also embed the Stellarium window.

2.3. Conceptual Telescope Control Hardware

While the physical hardware is conceptual for this paper, the software is designed to interface with a typical DIY motorized Alt-Azimuth mount.

Mechanical Design: Figure 2 illustrates a conceptual mechanical design for an Alt-Azimuth mount suitable for motorization. This could involve, for example, a Dobsonian-style base or a fork mount, with provisions for stepper motors and gear reduction on both axes [7].

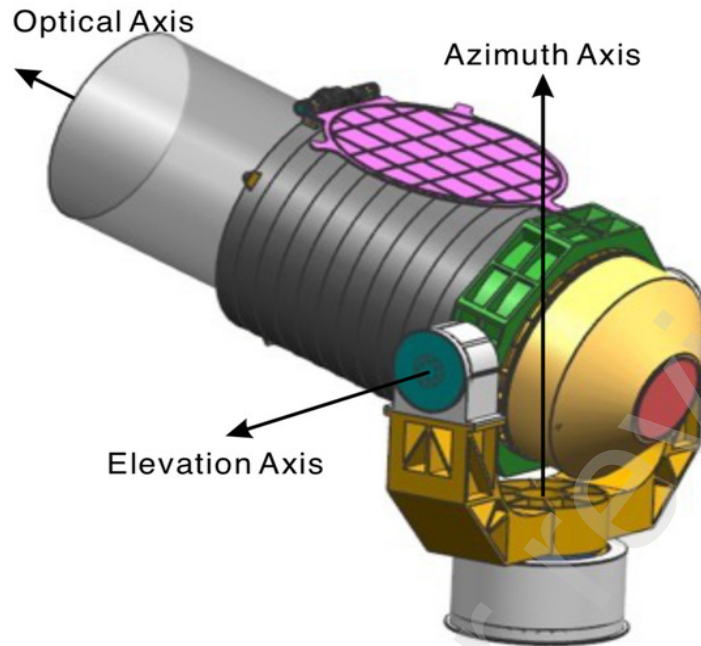


FIGURE 2: Conceptual mechanical design of the motorized Alt-Azimuth telescope mount.

- **Electronic Control System:** Figure 3 A microcontroller (e.g., Arduino Uno/Mega, ESP32) receives commands from InterstellarTracker. It uses motor driver ICs (e.g., DRV8825) to control stepper motors, which provide precise angular positioning. The firmware on the MCU is responsible for translating received Alt/Az commands into motor steps.

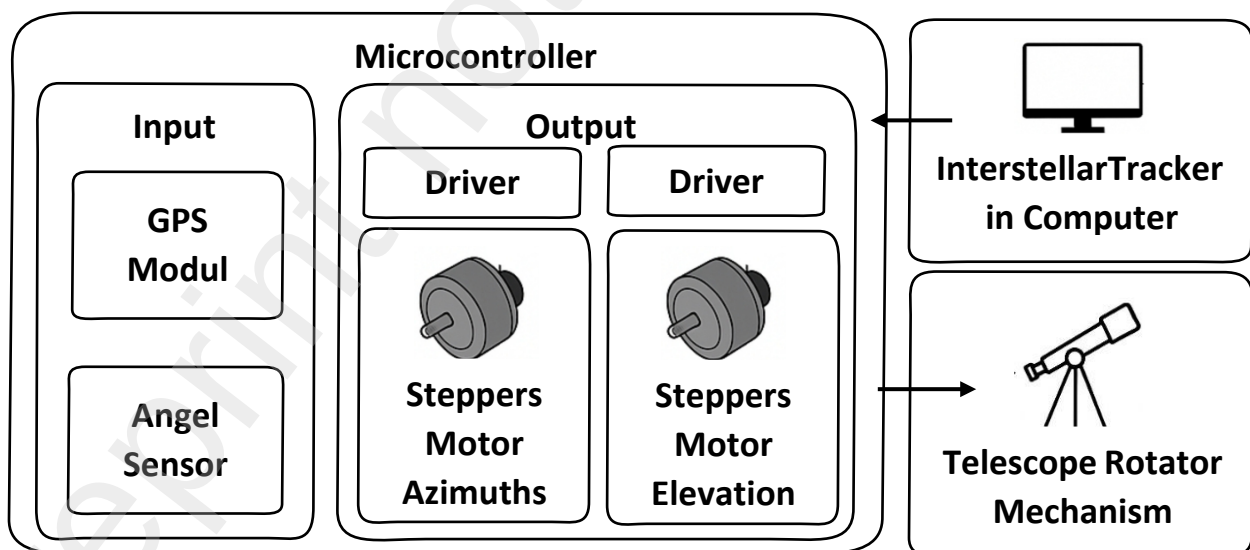


FIGURE 3: Device-level block diagram of a typical microcontroller-based telescope mount control system.

3. Software Implementation

The InterstellarTracker software is implemented in Python 3, utilizing several key libraries. The main application window and control logic are built using PyQt5. Asynchronous communication with Stellarium is handled by a custom ApiWorker (QThread-based) class using the requests library. The BoardManager class handles abstracted communication to the hardware.

3.1. Stellarium API Communication

Communication with Stellarium's Remote Control plugin is achieved via HTTP GET and POST requests to its local API endpoint (typically `http://localhost:8090/api/`).

- **Data Acquisition:** To obtain the current Alt/Az coordinates of the object selected in Stellarium, InterstellarTracker periodically polls the `/api/objects/info` endpoint, requesting data for the object currently in focus in Stellarium. Parameters like `name=@current` or `name=@focus` can be used.
- **Data Formats:** Stellarium returns data in JSON format. The software parses this JSON to extract relevant fields such as azimuth and altitude (Stellarium's term for elevation).
- **Rate Control:** Polling frequency is managed by a QTimer within InterstellarTracker, typically set between 1 to 5 seconds, to balance tracking responsiveness with API load.

3.2. Control Algorithm Design (Alt-Az Mount)

The primary control logic for an Alt-Azimuth mount is straightforward once target Alt/Az coordinates are obtained from Stellarium:

1. **Target Acquisition:** InterstellarTracker retrieves the current topocentric Azimuth (`Az_target`) and Elevation (`El_target`) of the selected object from Stellarium.
2. **Command Generation:** These `Az_target` and `El_target` values are directly used as the desired pointing angles for the telescope mount. No complex coordinate transformations are required by InterstellarTracker itself if Stellarium provides local Alt/Az.
3. **Transmission:** The target angles are formatted into a command string and sent via the BoardManager to the microcontroller.
4. **Microcontroller Action:** The microcontroller firmware receives these target Alt/Az values. It compares them to the mount's current (known or assumed) Alt/Az position and calculates the required motor steps for each axis to slew to the new target. This involves:
 - Knowing the steps per degree for each motor axis (determined by motor step angle, microstepping, and gear ratio).
 - Implementing acceleration/deceleration profiles for smooth motor movement (e.g., using the AccelStepper library on Arduino).
5. **Tracking:** This process (Steps 1-4) repeats at the defined update interval, causing the mount to incrementally follow the object's apparent motion across the sky.

Handling Limits: Software limits for azimuth (e.g., 0-360 degrees or a range to avoid cable wrap) and elevation (e.g., 0-90 degrees or practical limits of the mount) are notionally enforced by the microcontroller firmware based on its calibration and understanding of the physical mount.

InterstellarTracker sends the raw target coordinates from Stellarium; the firmware should validate these against mechanical limits.

3.3. Command String Structure (Serial/UART Protocol)

A simple, human-readable ASCII string protocol is proposed for UART communication between InterstellarTracker and the microcontroller, prioritizing ease of implementation on the MCU side.

- **Example Command Format:** AZ:target_azimuth,EL:target_elevation\n
 - AZ:: Prefix for Azimuth value.
 - target_azimuth: Floating-point number representing degrees (e.g., 120.5).
 - EL:: Prefix for Elevation value.
 - target_elevation: Floating-point number representing degrees (e.g., 45.0).
 - \n: Newline character to terminate the command.
- **Example:** "AZ:120.50,EL:45.00\n" This format is implemented in the _send_uart method of the BoardManager class.

3.4. User Interface

The InterstellarTracker GUI (**Figure 4**, previously described) allows users to:

- Configure hardware parameters (board type, port, model, baud rate).
- Initiate and terminate the connection to the telescope control hardware.
- Start and stop the tracking loop, which polls Stellarium for the currently selected object's coordinates and sends them to the hardware.
- View application logs and status messages.
- On Windows, the Stellarium window can be embedded within the InterstellarTracker application for a unified display. Otherwise, users interact with a separate Stellarium window for object selection and sky visualization.

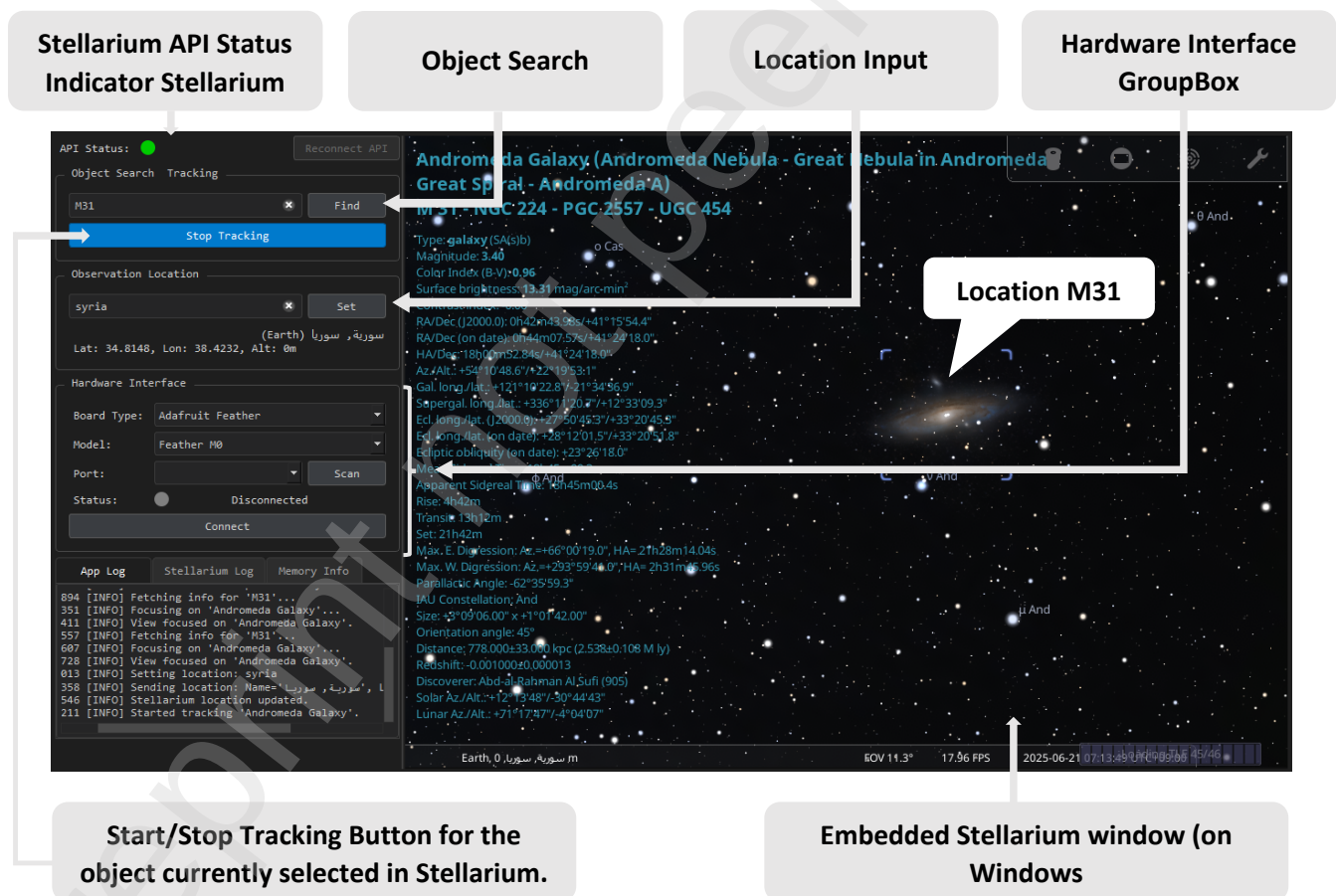


FIGURE 4: Screenshot of the InterstellarTracker main user interface.

4. Simulation and Testing

As the physical telescope mount hardware is conceptual at this stage, the functionality of the InterstellarTracker software, particularly its ability to acquire data from Stellarium and generate correct pointing commands, was tested through simulation.

- **Test Environment:**

- InterstellarTracker Python application running on Windows 10.
- Stellarium v0.2x.x (or latest stable) with the Remote Control plugin enabled and configured.
- A serial port monitoring tool (e.g., PuTTY, RealTerm, or a Python script emulating the microcontroller's serial input) was used to capture the command strings output by InterstellarTracker's BoardManager via a virtual serial port pair or by connecting to a simple echo device.

- **Test Cases:**

- **Static Target Acquisition:**

- An object (e.g., Polaris, a bright star, a planet) was selected in Stellarium.
- InterstellarTracker was commanded to "Start Tracking."
- The Alt/Az coordinates reported by Stellarium for the object were compared with the AZ:...EL:...\\n command strings logged by the serial monitor.

- **Dynamic Target Tracking (Simulated Slew):**

- A relatively fast-moving object (e.g., the Moon, or by manually slewing Stellarium's view rapidly) was selected.
- The sequence of command strings output by InterstellarTracker was observed to ensure they reflected the changing Alt/Az coordinates from Stellarium at the configured update interval.
 - **Boundary Conditions:** Targets near zenith and horizon were selected in Stellarium to verify command generation across a wide range of Alt/Az values.

- **Simulation Results:**

- InterstellarTracker successfully connected to the Stellarium API and retrieved object information.
- The BoardManager correctly formatted and output command strings via UART (simulated). **Figure 5** shows an example **JSON** from InterstellarTracker (or the serial monitor) displaying the generated command strings corresponding to Stellarium's reported Alt/Az for a selected target.
- The output Alt/Az values in the command strings consistently matched (within floating-point precision) the topocentric Alt/Az values displayed by Stellarium for the selected object at the moment of the API query.
- The command update rate was stable at the configured interval (e.g., every 2 seconds).

```

1 {
2   "above-horizon": true,
3   "airmass": -1,
4   "altitude": 11.5878542126841,
5   "altitude-geometric": 11.42034989663635,
6   "ambientInt": 0.0499997839331627,
7   "ambientLum": 0.0010000000474974513,
8   "appSidm": "17h36m21.7s",
9   "appSidm-dd": 264.09051676010597,
10  "axis-major": 0.055006963976016975,
11  "axis-major-dd": 3.1516668796539307,
12  "axis-major-deg": "3.15167°",
13  "axis-major-dms": "+3°09'06.00\"",
14  "axis-minor": 0.01794780200307358,
15  "axis-minor-dd": 1.028333306312561,
16  "axis-minor-deg": "1.02833°",
17  "axis-minor-dms": "+1°01'42.00\"",
18  "azimuth": 47.0559137467392,
19  "azimuth-geometric": 47.0559137467392,
20  "bv": 0.9600000381460727,
21  "bmag": 4.360000133514404,
22  "dec": 41.40507104326225,
23  "decJ2000": 41.26513916429588,
24  "designations": "M 31 - NGC 224 - PGC 2557 - UGC 454",
25  "ecliptic-obliquity": 23.43596523010362,
26  "elat": 33.34876686507645,
27  "elatJ2000": 33.3458605349535,
28  "elong": 28.199365781152654,
29  "elongJ2000": 27.84612262256837,
30  "found": true,
31  "glat": -21.576884219985146,
32  "glong": 121.17316950000064,
33  "hourAngle-dd": 253.05862574573513,
34  "hourAngle-hms": "16h52m14.1s",
35  "iauConstellation": "And",
36  "localized-name": "Andromeda Galaxy",
37  "meanSidm": "17h36m21.6s",
38  "meanSidm-dd": 264.09001490241644,
39  "morpho": "SA(s)b",
40  "name": "Andromeda Galaxy",
41  "object-type": "galaxy",
42  "orientation-angle": 45,
43  "parallacticAngle": -53.30635481133709,
44  "ra": 11.031801014370828,
45  "raJ2000": 10.683459395958753,
46  "redshift": -0.0010000000474974513,
47  "rise": "4h34m",
48  "rise-dhr": 4.569999999999999,
49  "set": "21h34m",
50  "set-dhr": 21.56638888888889,
51  "sglat": 12.55241159973071,
52  "sglong": 336.1891576607911,
53  "size": 0.055006963976016975,
54  "size-dd": 3.1516668796539307,
55  "size-deg": "3.15167°",
56  "size-dms": "+3°09'06.00\"",
57  "surface-brightness": 13.305166244506836,
58  "transit": "13h04m",
59  "transit-dhr": 13.060055555555555,
60  "type": "galaxy",
61  "vmag": 3.4000000053674316,
62  "vmax": 3.4000000053674316,
63  "_last_updated_utc": "2025-06-22T20:57:30.063704Z"
64 }

```

Figure 5 shows an example of the JSON data retrieved from Stellarium for a target like the Andromeda Galaxy, from which InterstellarTracker extracts the necessary pointing coordinates (e.g., 'azimuth', 'altitude'). The generated command strings based on such data were then verified.

These simulation results validate the core software functionality: accurate data acquisition from Stellarium and correct command string generation according to the defined protocol, forming a solid software foundation for controlling a physical automated telescope mount.

5. Discussion

The InterstellarTracker software offers a robust and flexible framework for developing Stellarium-integrated automated telescope control systems. Its design prioritizes modularity and the use of established open-source tools.

• Comparison with Existing Systems:

- Unlike monolithic commercial GoTo systems, InterstellarTracker decouples the "sky intelligence" (Stellarium) from the low-level hardware control, allowing for greater flexibility in choosing or designing the physical mount and electronics.
- While ASCOM provides a comprehensive standard, InterstellarTracker aims for a simpler direct control pathway for custom/DIY setups, potentially lowering the barrier to entry for automation. The BoardManager concept allows for this directness while retaining structure.
- Its reliance on Python makes it accessible for customization and integration with other scientific Python libraries, **potentially including those used for image processing, astrometry, or orbit determination in astronomical contexts.**

• Advantages of Stellarium API and Modular Control Logic:

- **Accurate Ephemerides:** Stellarium provides a vast and continuously updated database of celestial objects and precise positional calculations, **which is foundational for any tracking application, including those supporting spacecraft navigation or NEO detection.**
- **User-Friendly Sky Navigation:** Users benefit from Stellarium's intuitive graphical interface for target selection.
- **Modular Hardware Interface:** The BoardManager allows the core InterstellarTracker software to remain largely independent of the specific microcontroller or communication nuances of the hardware backend. This facilitates adaptation to different hardware configurations.
- **Potential Integration with Motorized Mounts and Relevance to Astronautics:** The generated AZ/EL command strings are directly usable by microcontroller firmware (e.g., on Arduino or ESP32 platforms) employing stepper motors for an Alt-Azimuth mount. The conceptual mechanical designs presented in **Figure 2** provide a starting point for realizing such a physical system. The next step would involve developing firmware to parse these commands and implement precise motor control and calibration routines.

From an astronomical perspective, a network of such cost-effective, automated telescopes could contribute to:

- **Space Situational Awareness (SSA):** Detecting and tracking satellites or space debris, especially if equipped with appropriate imaging sensors and coupled with astrometric analysis software. The system's adaptability allows for experimentation with different optical tube assemblies (OTAs) optimized for such tasks.
- **Near-Earth Object (NEO) Surveys and Follow-up:** Contributing to the discovery or confirmation of NEOs by enabling rapid pointing to candidate objects identified by larger survey telescopes.
- **Educational Outreach and Citizen Science:** Providing accessible tools for students and citizen scientists to engage in real astronomical observations, fostering interest in space science and engineering.
- **Validation of Ground Support Technologies:** Serving as a testbed for developing and validating control algorithms, sensor integration techniques (e.g., for pointing correction or weather monitoring), and remote operation protocols that could be scaled to more professional ground station equipment.
- **Optical Communication Ground Terminal Pointing:** While requiring significantly higher precision and dedicated hardware, the foundational software principles for automated pointing share commonalities with those needed for establishing optical links with spacecraft, making this a relevant starting point for educational exploration of such concepts.

The development of robust calibration routines and, eventually, closed-loop feedback mechanisms (as outlined in Future Work) would be critical for achieving the pointing accuracies required for many of these astronomical support applications.

6. Limitations and Future Work

6.1. Current Limitations

- **Conceptual Hardware Base:** The most significant limitation is that the physical telescope mount and its control electronics are currently conceptual. The performance of the complete system (pointing accuracy, tracking stability) can only be fully evaluated with a physical implementation.
- **Open-Loop Control:** The current software design assumes an open-loop control system at the hardware level. It sends target coordinates but does not receive feedback on the telescope's actual position from encoders or other sensors.
- **Potential Network Latency/API Reliability:** Relies on the responsiveness of the local Stellarium API. While generally fast for local communication, any significant delays or API unavailability would impact tracking performance.
- **Calibration Dependence:** The accuracy of a physical system will be highly dependent on the quality of the initial mount calibration (e.g., star alignment) performed by the user and managed by the microcontroller firmware.

6.2. Planned Developments

Future work will focus on bridging the gap between the current software implementation and a fully functional physical system, as well as enhancing software capabilities:

- **Physical Base Prototyping:** Construction and testing of a motorized Alt-Azimuth mount based on the conceptual designs (**Figure 2**), incorporating stepper motors, drivers, and a chosen microcontroller (**Figure 3**).
- **Microcontroller Firmware Development:** Writing and refining the MCU firmware to:
 - Reliably parse AZ/EL commands from InterstellarTracker.
 - Implement precise stepper motor control with acceleration/deceleration.
 - Incorporate robust multi-star alignment routines for accurate calibration.
- **Integration of Angle Sensors:** Adding rotary encoders or other angle sensors to the mount axes to provide closed-loop feedback. This would allow the system to correct for motor step loss, mechanical slippage, and improve overall pointing and tracking accuracy. InterstellarTracker would be extended to receive and potentially utilize this feedback.
- **Advanced Pointing Models:** Incorporating more sophisticated pointing models in the firmware or InterstellarTracker to compensate for systematic errors (e.g., cone error, atmospheric refraction at a higher level).
- **Equatorial Mount Support:** Extending the control logic and BoardManager communication to support equatorial mounts, including handling sidereal tracking rates and RA/Dec coordinate systems more directly.
- **Automated Calibration Sequences:** Potentially using Stellarium scripting capabilities, initiated by InterstellarTracker, to guide the user through a more automated calibration process.

7. Software Availability

The InterstellarTracker Python source code is intended to be available for academic, educational, and research purposes. Details or access to the development repository may be provided by the corresponding author upon reasonable request, subject to ongoing development.

8. Acknowledgements

The developers of the Stellarium planetarium software are gratefully acknowledged for their outstanding contribution to the astronomy community and for providing the Remote Control plugin API. The Skyfield and PyQt5 libraries, and the broader Python open-source ecosystem, were essential for this development.

9. References

- [1] Chromey, F. R. (2010). *To Measure the Sky: An Introduction to Observational Astronomy*. Cambridge University Press.
 - [2] Ratledge, D. (Ed.). (2005). *The Art and Science of CCD Astronomy*. Willmann-Bell.
 - [3] Stellarium Planetarium Software & Remote Control Plugin Documentation. [Online]. Available: <https://stellarium.org/> and <https://stellarium.org/doc/head/remoteControlApi.html>
 - [4] Rhodes, B. (2019). Skyfield: High-precision astronomy for Python. [Online]. Available: <https://rhodesmill.org/skyfield/>
 - [5] Riverbank Computing. PyQt5. [Online]. Available: <https://www.riverbankcomputing.com/software/pyqt/>
 - [6] ASCOM Initiative. Astronomy Common Object Model. [Online]. Available: <https://ascom-standards.org/>
 - [7] Zhao, Y.; Wang, Y.; He, F.; Yao, Z.; Rong, Z.; Wei, Y. A Simplified Pointing Model for Alt-Az Telescopes. Appl. Sci. 2023, 13, 11238. <https://doi.org/10.3390/app132011238>
-

APPENDIX A: Conceptual Microcontroller Firmware - Functional Overview (for Automated Telescope Control)

The example microcontroller firmware, designed for platforms like Arduino, is responsible for receiving pointing commands from the host InterstellarTracker software, managing local hardware (motors, angle sensors, GPS, display), performing automated homing/alignment sequences, and executing the telescope pointing and tracking. It is structured around several key functional blocks:

A.1. System Initialization (setup()):

- **Communication Setup:** Configures serial communication interfaces for both the host PC (InterstellarTracker) and an attached GPS module.
- **Peripheral Initialization:**
 - Initializes output pins for stepper motor control (step, direction, enable if used) and status LEDs (Power, GPS Lock, Status, Aligned, Error).
 - Sets up the LCD display for user feedback.
 - **Angle Sensor Initialization:** (Conceptual) Executes any necessary setup routines for the specific angle sensors used (e.g., I2C/SPI bus configuration, encoder mode setting).
- **Motor Configuration:** Configures stepper motor instances (e.g., using a library like AccelStepper) with maximum speed and acceleration parameters, typically derived from desired degrees-per-second performance and mechanical properties (STEPS_PER_DEGREE).
- **Persistent Data Loading (loadPersistentData()):** Loads data from non-volatile memory (e.g., EEPROM), such as:
 - System alignment status (isAlignedAndHomed flag).
 - Stored offsets for incremental encoders if previously homed and calibrated.
- **Initial State Transition:** Transitions the system to an initial operational state based on loaded data (e.g., AWAITING_GPS_FIX if not previously aligned, or directly to ALIGNED_IDLE or AUTO_ALIGNING if alignment data exists and sensors are absolute/reliable on startup).

A.2. Main Operational Loop (loop()):

- **GPS Data Acquisition (readGpsSerial()):** Periodically reads and processes NMEA sentences from an attached GPS module to update observer location (latitude, longitude, altitude) and time, and to determine GPS lock status.
- **Angle Sensor Data Acquisition (readAngleSensors()):** Continuously reads data from the azimuth and elevation angle sensors to obtain the telescope's current actual orientation (currentActualAzimuthDeg, currentActualElevationDeg). This is the ground truth for the system's pointing.
- **Host Communication (readHostSerial()):** Checks for and processes incoming serial commands (e.g., AZ:target_azimuth,EL:target_elevation\n) from InterstellarTracker.
- **State Machine Execution (updateStateMachine()):** Manages system behavior and transitions between states based on current conditions (GPS lock, alignment status, host commands, sensor readings, motor status).
- **Motor Actuation:** Based on the current state and target coordinates:
 - If aligned and a new target is received, commands the stepper motors to move towards the target azimuth and elevation. This may involve open-loop commands to AccelStepper.moveTo() or, in a more advanced closed-loop system, direct speed control based on the error between target and actual sensor readings.
 - Continuously calls the run/update functions of the stepper motor library (e.g., azimuthStepper.run()).
- **Feedback Updates:**
 - Updates an LCD display (updateLcdDisplay()) with current status, actual/target coordinates, GPS info, etc.
 - Updates status LEDs (updateLedIndicators()) to reflect the system state.

A.3. Host Communication and Command Parsing (readHostSerial(), parseHostInputString()):

- Listens for newline-terminated command strings from InterstellarTracker.
- Parses these strings (e.g., "AZ:123.45,EL:67.89\n") to extract the target azimuth and elevation.
- Validates and constrains the received target angles against pre-defined operational limits of the mount.
- Updates internal target variables (targetAzimuthDeg, targetElevationDeg).
- Manages a timeout for host communication; if no commands are received for a defined period, the system may revert to an ALIGNED_IDLE state or hold its current position.

A.4. GPS Data Acquisition (readGpsSerial()):

- Continuously polls the GPS module for NMEA sentences.
- Uses a GPS parsing library (e.g., TinyGPS++) to decode valid location (latitude, longitude, altitude) and time data.
- Updates global variables for observer coordinates and GPS lock status.
- Manages GPS signal loss and timeout conditions, potentially triggering state changes if GPS is critical for ongoing operations or re-alignment.

A.5. Angle Sensor Interface (readAngleSensors(), conceptual setupAngleSensors()):

- **Initialization (setupAngleSensors()):** Performs any one-time setup required by the specific angle sensor hardware (e.g., setting I2C/SPI parameters, configuring encoder modes).
- **Data Reading (readAngleSensors()):** Contains the low-level hardware-specific code to read the current angular position from both azimuth and elevation sensors. This could involve:
 - Reading I2C/SPI registers for absolute encoders.
 - Counting pulses and direction for incremental encoders.
 - Reading analog values for potentiometers and converting to degrees.
 - Fusing data from an IMU.
- Updates currentActualAzimuthDeg and currentActualElevationDeg with the direct feedback from these sensors.

A.6. State Management (updateStateMachine()):

- Implements a finite state machine to control the overall behavior of the controller. Key states include:
 - INITIALIZING: Initial setup phase.
 - AWAITING_GPS_FIX: Waiting for a valid GPS fix, often a prerequisite for automatic alignment.
 - HOMING_AZIMUTH/HOMING_ELEVATION: (If using incremental encoders) Actively moving motors to find a known physical reference point (e.g., limit switch or index pulse).
 - AUTO_ALIGNING: After GPS lock and/or homing, this state uses sensor data to establish the telescope's absolute orientation with respect to the sky or a known terrestrial reference (e.g., True North, Horizon). For absolute sensors, this might be a quick read-and-set. For homed incremental systems, it confirms the homed position.

- ALIGNMENT_FAILED: Entered if the automatic alignment process cannot complete successfully.
- ALIGNED_IDLE: System is successfully aligned/homed, GPS locked (if used for continuous time/location), and awaiting commands or has reached its last target.
- SLEWING: Actively moving motors significantly to a new target.
- TRACKING: Actively making (typically smaller) adjustments to follow a celestial object based on commands from InterstellarTracker.
- ERROR_STATE: Indicates a critical system fault, usually stopping motors.
- Transitions between states are triggered by events such as GPS lock acquisition, completion of homing/alignment, reception of new tracking commands, host communication timeout, or sensor errors.

A.7. Motor Control and Position Conversion:

- Utilizes a stepper motor control library (e.g., AccelStepper) to manage smooth acceleration, deceleration, and movement to target step positions.
- Includes functions to convert angular degrees (received from InterstellarTracker or calculated internally) to motor steps, based on the STEPS_PER_DEGREE_AZ and STEPS_PER_DEGREE_EL constants reflecting the telescope's mechanical properties.
- Conversely, converts current motor step positions (the stepper library's internal count) back to estimated degrees (currentStepperAzimuthDeg, currentStepperElevationDeg) for display or open-loop reference.
- In a closed-loop system, motor commands would be adjusted based on the error between target...Deg and currentActual...Deg (from sensors).

A.8. Calibration, Homing, and Persistent Storage (runAlignmentRoutine() - now implicit in auto-align states, loadPersistentData(), markSystemAligned(), saveSensorOffsetsIfNecessary()):

- **Automatic Homing/Alignment:** The HOMING_... and AUTO_ALIGNING states manage the process of establishing the telescope's absolute orientation.
 - This may involve driving motors to limit switches or index pulses (for incremental encoders).
 - For absolute sensors, it involves reading their initial state and potentially applying stored calibration offsets.
 - The outcome is that the system knows how its internal sensor readings (and thus motor step counts) relate to a real-world coordinate system (e.g., Az=0 is True North, El=0 is Horizon).
- **Persistent Storage:**
 - Functions to save and load alignment status (isAlignedAndHomed flag) to/from non-volatile memory (e.g., EEPROM).
 - (Conceptual) Functions to save and load sensor calibration offsets or homing reference points (saveSensorOffsetsIfNecessary()). This ensures the system can resume operation after a power cycle with minimal or no re-calibration if conditions permit (e.g., if absolute sensors are used or the mount wasn't significantly disturbed).

A.9. User Interface Updates (updateLcdDisplay(), updateLedIndicators()):

- Periodically refreshes an LCD screen to display crucial information such as current system state, GPS status, actual sensor-derived azimuth/elevation, target azimuth/elevation, and error messages.
- Controls status LEDs to provide quick visual feedback (e.g., GPS lock, alignment status, motor activity, error).

APPENDIX B: InterstellarTracker Python Application - Internal Workings

This appendix provides a high-level overview of the internal architecture and operational flow of the InterstellarTracker Python application. It is intended to give a general understanding of how the software interfaces with Stellarium and manages communication with the conceptual telescope control hardware, rather than a detailed line-by-line code analysis.

B.1. Core Technologies and Structure

- **Language and Libraries:** The application is written in Python 3. Key libraries include:
 - **PyQt5:** For creating the Graphical User Interface (GUI), managing application events, and structuring the main application window (class `StellariumTracker`, inheriting from `QWidget` or `QMainWindow`).
 - **Requests:** For making HTTP requests to the Stellarium Remote Control API.
 - **Skyfield (Optional/Auxiliary):** While primary celestial data comes from Stellarium, Skyfield might be used for auxiliary calculations like detailed ground track visualization for satellites (if that feature from a previous context were adapted), or advanced coordinate transformations not directly provided by the Stellarium API if needed in future enhancements. The current code primarily uses it for timescale (`ts = load.timescale()`).
 - **psutil:** For system memory monitoring.
 - **Serial (pyserial):** For UART communication with the telescope microcontroller via the BoardManager. Foundational support for I2C (`smbus2`) and SPI (`spidev`) is included, typically for Linux-based systems, with mock implementations for other platforms or if libraries are missing.
 - **Platform-specific libraries (e.g., pywin32, winreg on Windows):** Used for features like embedding the Stellarium window within the InterstellarTracker GUI and for more robust serial port detection on Windows.
 - **Standard Python libraries:** `json` (for API responses), `os`, `sys`, `time`, `logging`, `threading` (via `QThread`), `re`, `urllib.parse`.
- **Main Application Class (StellariumTracker):** This class, likely inheriting from `QWidget` or `QMainWindow`, serves as the central hub of the application. It:
 - Initializes the User Interface (UI).
 - Manages connections between UI elements (buttons, inputs) and backend logic (Python functions/slots).
 - Coordinates communication with the Stellarium API.
 - Instantiates and manages the BoardManager for hardware communication.
 - Handles application state (e.g., connecting to Stellarium, tracking active).
 - Manages timers for periodic tasks.

B.2. User Interface (GUI) Management

- **UI Definition:** The GUI is constructed using PyQt5 widgets (e.g., QPushButton, QLineEdit, QComboBox, QTextEdit, QTabWidget, QGroupBox). The layout is typically defined in an `init_ui` method, organizing elements into logical groups for Stellarium interaction, hardware configuration, and information display.
- **Signal-Slot Mechanism:** PyQt's signal-slot mechanism is used extensively to connect user actions (e.g., button clicks, text changes in input fields, combo box selections) to corresponding Python functions (slots) that handle the application logic.
- **Stellarium Window Embedding (Windows-specific):** If on Windows and `pywin32` is available, the application attempts to find the Stellarium main window and reparent it into a designated QWidget container within the InterstellarTracker GUI. This uses Windows API calls for window manipulation (`SetParent`, `MoveWindow`, etc.) and may involve a `WinEventHook` to detect Stellarium window creation.
- **Dynamic Updates:** QTimer objects are used to periodically trigger updates to:
 - The telescope's target coordinates (by querying Stellarium when tracking is active).
 - Status displays (e.g., API connection status, hardware connection status).
 - System memory information.
 - Log displays.

B.3. Stellarium API Communication and Data Handling

- **API Interaction (ApiWorker class):** Asynchronous communication with Stellarium's HTTP API is managed by the ApiWorker class, which inherits from QThread. This prevents blocking the main GUI thread during network requests.
 - Each API request (e.g., get object info, set location) is typically handled by a new ApiWorker instance.
 - It uses the requests library to perform GET or POST operations.
 - Handles API authentication (if configured in Stellarium, though the default password is often used for local connections).
 - Emits finished (with data) or error signals upon completion.
- **Key API Endpoints Used:**
 - `/api/main/status`: To check API health and get current observer location/time.
 - `/api/objects/info?name=<object_name_or_@focus>`: To retrieve detailed information (including Alt/Az, RA/Dec) for the currently selected object in Stellarium.
 - `/api/main/focus` (POST): To command Stellarium to select and center on a specified target.
 - `/api/location/setlocationfields` (POST): To set Stellarium's observer location.
 - `/api/stelaction/do` (POST): To trigger Stellarium actions (e.g., toggle atmosphere, grids).
 - `/api/scripts/run` (POST): To execute custom scripts within Stellarium (e.g., for placing markers).
- **Data Parsing:** Stellarium API responses are typically in JSON format, which are parsed into Python dictionaries.
- **Coordinate Acquisition:** When tracking is active, a timer triggers a request to `/api/objects/info` for the object currently focused in Stellarium. The returned Azimuth and Elevation values are then used for commanding the telescope mount.

B.4. Hardware Communication (BoardManager class)

- **Abstraction Layer:** The BoardManager class (inheriting from QObject) is designed to abstract the details of communicating with different types of microcontrollers or hardware interfaces.
- **Supported Protocols:**
 - **UART (Serial):** The primary and most developed method. Sends formatted ASCII strings (e.g., "AZ:xxx.xx,EL:yyy.yy\n") over a selected serial port using pyserial.
 - **I2C & SPI:** Foundational support is present, primarily intended for Linux systems where smbus2 (for I2C) and spidev (for SPI) are available. Mock implementations are provided for other platforms or if the libraries are not installed, allowing the software to run but without actual I2C/SPI hardware communication.
- **Board Definitions:** A dictionary (BoardManager.boards) stores predefined configurations for different board types, including their expected communication protocol, and model-specific details like I2C addresses or SPI bus/device numbers.
- **Port Management:** Includes functions to scan for available serial ports using platform-specific methods (_windows_board_check, _linux_board_check, _macos_board_check).
- **Connection Lifecycle:** Manages connecting to and disconnecting from the selected hardware port and board configuration. Emits a connection_changed signal to update the GUI.
- **Data Sending:** The send_data(data: dict) method takes a dictionary (e.g., {'azimuth': 120.5, 'elevation': 45.0}) and formats it according to the active protocol before sending it to the connected hardware.

B.5. Control Logic and Tracking Loop

- **Tracking Activation:** A "Start Tracking" button in the GUI toggles the tracking_active flag.
- **Main Tracking Timer:** When tracking_active is true, a QTimer (track_timer) periodically triggers the safe_update_tracking function.
- **safe_update_tracking():** This function:
 1. Calls get_object_info(self.current_object) where self.current_object is the name of the object currently selected/focused in Stellarium.
 2. get_object_info in turn calls the Stellarium API to fetch the latest coordinates.
- **handle_object_info_response():** This slot receives the data from Stellarium.
 1. It extracts Azimuth and Elevation (Stellarium's altitude field is used for elevation).
 2. If the BoardManager is connected to hardware, it constructs a data dictionary ({'azimuth': az, 'elevation': el}) and calls self.board_manager.send_data() to transmit the pointing command.
 3. It may also update auxiliary visualizations (e.g., markers for sub-object points if Skyfield calculations are involved, though this is less central for direct telescope pointing via Stellarium's Alt/Az).

B.6. Threading and Asynchronous Operations

- **ApiWorker (QThread):** As mentioned, all Stellarium API calls are made in separate threads to keep the GUI responsive.
- **Timers (QTimer):** Used for periodic polling (tracking updates, memory info, log watching) without blocking the main event loop.

B.7. Configuration and Settings

- User preferences (e.g., last used hardware configuration, window size/position, observer location if manually entered into InterstellarTracker) are likely intended to be saved and loaded, potentially using PyQt's QSettings mechanism or a custom JSON configuration file, allowing settings to persist between sessions.
- Default Stellarium API URL and password constants are defined within the script.
- Stellarium's own config.ini may be checked and modified (on Windows with admin rights) to ensure the Remote Control plugin is enabled.

B.8. Logging and Error Handling

- The application implements logging using Python's logging module, writing to both a file (Interstellar_tracker.log) and a QTextEdit widget in the GUI's "App Log" tab.
- API errors from ApiWorker are signaled and can be displayed to the user (e.g., in the log or via message boxes).
- Hardware connection errors from BoardManager are also signaled and update the GUI.
- A global exception hook (sys.excepthook) is set up to catch unhandled exceptions, log them, and attempt to display a critical error message to the user.

B.9. Main Operational Flow (Simplified)

1. Initialization:

- Application starts, StellariumTracker instance created.
- UI is initialized and displayed.
- (Intended) Saved settings are loaded.
- Attempts to launch/embed Stellarium (Windows-specific).
- Initiates connection to Stellarium API.

2. User Configuration:

- User sets observer location (typically via Stellarium, or can send command from InterstellarTracker).
- User configures hardware interface (board type, port, etc.) in InterstellarTracker and clicks "Connect."

3. Target Selection (in Stellarium):

- User navigates in Stellarium and selects a celestial object.

4. Tracking (initiated from InterstellarTracker):

- User clicks "Start Tracking" in InterstellarTracker.
- The main tracking timer is activated.
- On each timer tick:
 - InterstellarTracker queries Stellarium API for the current Alt/Az of the focused object.
 - The GUI's telemetry (if any for the telescope itself) and map (Stellarium's view) are effectively updated by Stellarium.
 - If hardware communication is active, the BoardManager sends the Az/EI data to the connected microcontroller.

5. Termination:

- User closes the application.

- The `closeEvent` method handles graceful shutdown: stops timers, disconnects hardware, waits for worker threads, resets Stellarium parent (if embedded), and optionally terminates a self-launched Stellarium process.